

ECOM EXPERTS

Ecom KPIs PrestaShop — version Firefox

Manual de la version Firefox e informe de la revision

ecom_kpis_prestashop-firefox 1.0.2 · 04/09/2026

Contenido

1. Qué hace y para quién

2. Compatibilidad

3. Instalación

4. Configuración

Las capturas

5. Cómo se usa, día a día

6. Qué ve el cliente en la tienda

7. Preguntas frecuentes y problemas

8. Seguridad

8.1 Punto de partida

8.2 Lo que se ha portado en esta revisión

8.3 Lo que NO se ha podido portar tal cual, y su equivalente en Firefox

8.4 Lo que queda pendiente, y por qué

8.5 Cómo se ha comprobado

9. Ficha técnica

Este manual es el complemento de la versión Firefox. La extensión es la misma herramienta que la de Chrome/Edge y comparte todo el código de la interfaz, así que aquí no se repite lo que ya está explicado. **Para saber qué hace, cómo se configura y cómo se usa día a día, lee el manual principal:**

`../ecom_kpis_prestashop/MANUAL.md` (y su PDF, `ecom_kpis_prestashop-manual.pdf`, en esa misma carpeta).

Lo que sigue es **solo lo propio de Firefox**: qué cambia respecto a la versión Chrome, qué se ha portado en esta revisión y qué diferencias de comportamiento hay que conocer.

1. Qué hace y para quién

Idéntico a la versión Chrome. Es una extensión que pone el cuadro de mandos de tu tienda PrestaShop a un clic del icono de la barra de Firefox: ventas de hoy, de la semana y del mes, comparativa con el mes anterior, pedidos por estado, clientes nuevos, ticket medio, gráfico de los últimos siete días, stock bajo y últimos pedidos. Lee la **Webservice API** de PrestaShop en modo consulta y lo calcula todo dentro del navegador.

Explicación completa en la sección 1 del manual principal.

2. Compatibilidad

Esto **sí** cambia respecto a la versión Chrome:

	Versión Firefox	Versión Chrome
Navegador	Firefox 116 o superior	Chrome/Edge 102 o superior
Manifiesto	MV3 con <code>browser_specific_settings.gecko</code>	MV3
Pantalla de opciones	<code>options_ui</code> con <code>open_in_tab: true</code>	<code>options_page</code>
Identificador AMO	<code>ecom-kpis-prestashop@ecomexperts.es</code>	—
Tienda	PrestaShop 1.6 en adelante, Webservice activa y con salida JSON	igual

Por qué Firefox 116 y no 115, que era el mínimo hasta ahora. La corrección más importante de esta revisión sustituye el permiso permanente a todos los dominios por `optional_host_permissions`, y esa clave del manifiesto **no existe en Firefox antes de la 116**: en Firefox 115 el manifiesto se aceptaría, la clave se ignoraría en silencio y la extensión se quedaría sin poder llamar a ninguna tienda. Por eso `strict_min_version` sube a `116.0`. Firefox 115 es la ESR antigua; quien siga ahí tiene que actualizar.

No hay multitienda ni multiidioma: la interfaz está en castellano y la clave de la Webservice se asocia a una sola tienda.

3. Instalación

Desde **addons.mozilla.org**, con el botón «Añadir a Firefox», o cargando el `.xpi` firmado. Para probarla sin firmar: `about:debugging` → **Este Firefox** → **Cargar complemento temporal** → elegir el `manifest.json` de la carpeta.

La extensión **no crea nada** en tu tienda ni en tu equipo: no toca la base de datos de PrestaShop, no instala ningún módulo y no escribe ficheros. Todo lo que guarda son cuatro valores (URL, clave, umbral de stock y símbolo de

moneda) en el almacenamiento local de Firefox, más el permiso de host que le concedas. Al desinstalarla, Firefox borra las dos cosas y no queda rastro.

4. Configuración

La pantalla es la misma que en Chrome y está explicada campo a campo en la sección 4 del manual principal. En Firefox hay **dos diferencias que conviene conocer**:

1. La pantalla se abre en una pestaña propia, no dentro de `about:addons`. El manifiesto declara `options_ui.open_in_tab: true` a propósito. No es estética: dentro del panel incrustado de `about:addons`, Firefox no llega a mostrar el aviso de permisos, así que pulsar **Guardar** no serviría de nada. Ábrela siempre desde el icono de la extensión (🔌) o desde el botón «Configurar conexión» del popup.

2. El aviso de permiso lo pinta Firefox con su propio texto, y aparece justo al pulsar **Guardar** o **Probar conexión**, preguntando si permites que la extensión acceda a los datos de tu dominio. Hay que aceptarlo: sin ese permiso la extensión no puede llamar a la API y el popup te lo dirá.

Aparte de eso, el orden es el de siempre: activar la Webservice en PrestaShop, generar una clave de **solo lectura** con permisos GET sobre `orders`, `order_states`, `stock_availables`, `products`, `customers` y `shops`, y pegar aquí la URL (sin `/api`) y la clave.

Las capturas

⚠ **Las capturas que siguen son de la versión Chrome de la extensión.** Playwright no puede cargar complementos en su Firefox, así que no se han podido rehacer en Firefox sin el visto bueno para abrir un navegador. **El HTML y el CSS de las dos versiones son idénticos** — `popup/popup.html`, `popup/popup.css` y `options/options.html` no difieren en una sola línea salvo el texto de dos avisos—, de modo que lo que se ve aquí es lo que se ve en Firefox, con la decoración de ventana del otro navegador. Lo único que en Firefox se ve distinto es el **aviso de permiso**, que lo dibuja el propio navegador.

Pantalla de configuración recién abierta, sin nada guardado:

Ecom KPIs PrestaShop — Configuración

URL de la tienda

Sin `/api` al final. Ej: `https://mitienda.com`

Clave de la Webservice API

Ver

Se guarda solo en tu navegador (`chrome.storage.local`). Usa una clave de **solo lectura**: no hace falta darle permisos de escritura.

Umbral de stock bajo

Símbolo de moneda

GuardarProbar conexión

Cómo activar la Webservice en PrestaShop:

1. Parámetros avanzados → Webservice → Activar.
2. Añadir nueva clave → genera la clave.
3. Da permisos de **Ver (GET)** a: `orders`, `order_states`, `stock_avaliables`, `products`, `customers`, `shops`.
4. Guarda y pega aquí la URL y la clave.
5. Al pulsar **Guardar** el navegador pedirá permiso para acceder a ese dominio: acepta. La extensión solo puede leer el dominio que autorices aquí, ninguno más.

Con la clave enmascarada y el botón **Ver** al lado, y la conexión probada con éxito:

Ecom KPIs PrestaShop — Configuración

URL de la tienda

Sin `/api` al final. Ej: `https://mitienda.com`

Clave de la Webservice API

Ver

Se guarda solo en tu navegador (`chrome.storage.local`). Usa una clave de **solo lectura**: no hace falta darle permisos de escritura.

Umbral de stock bajo

Símbolo de moneda

GuardarProbar conexión

Conexión correcta. La clave lee pedidos.

Cómo activar la Webservice en PrestaShop:

1. Parámetros avanzados → Webservice → Activar.
2. Añadir nueva clave → genera la clave.
3. Da permisos de **Ver (GET)** a: `orders`, `order_states`, `stock_avaliables`, `products`, `customers`, `shops`.
4. Guarda y pega aquí la URL y la clave.
5. Al pulsar **Guardar** el navegador pedirá permiso para acceder a ese dominio: acepta. La extensión solo puede leer el dominio que autorices aquí, ninguno más.

Rechazo de una dirección `http://` de un dominio real:

Ecom KPIs PrestaShop — Configuración

URL de la tienda

Sin `/api` al final. Ej: `https://mitienda.com`

Clave de la Webservice API

[Ver](#)

Se guarda solo en tu navegador (`chrome.storage.local`). Usa una clave de **solo lectura**: no hace falta darle permisos de escritura.

Umbral de stock bajo

Símbolo de moneda

[Guardar](#)[Probar conexión](#)

Por seguridad la tienda debe usar `https://` (la clave de la API viajaría en claro).

Cómo activar la Webservice en PrestaShop:

1. Parámetros avanzados → Webservice → Activar.
2. Añadir nueva clave → genera la clave.
3. Da permisos de **Ver (GET)** a: `orders`, `order_states`, `stock_availabilitys`, `products`, `customers`, `shops`.
4. Guarda y pega aquí la URL y la clave.
5. Al pulsar **Guardar** el navegador pedirá permiso para acceder a ese dominio: acepta. La extensión solo puede leer el dominio que autorices aquí, ninguno más.

Guardado correcto:

Ecom KPIs PrestaShop — Configuración

URL de la tienda

Sin `/api` al final. Ej: `https://mitienda.com`

Clave de la Webservice API

Ver

Se guarda solo en tu navegador (`chrome.storage.local`). Usa una clave de **solo lectura**: no hace falta darle permisos de escritura.

Umbral de stock bajo

Símbolo de moneda

GuardarProbar conexión

Guardado.

Cómo activar la Webservice en PrestaShop:

1. Parámetros avanzados → Webservice → Activar.
2. Añadir nueva clave → genera la clave.
3. Da permisos de **Ver (GET)** a: `orders`, `order_states`, `stock_avaliables`, `products`, `customers`, `shops`.
4. Guarda y pega aquí la URL y la clave.
5. Al pulsar **Guardar** el navegador pedirá permiso para acceder a ese dominio: acepta. La extensión solo puede leer el dominio que autorices aquí, ninguno más.

5. Cómo se usa, día a día

Igual que en Chrome; está detallado en la sección 5 del manual principal. El cuadro de mandos completo:

Ecom KPIs

842,05 €

Ventas hoy

2143,17 €

Esta semana

2143,17 €

Este mes ▲ 268%

16

Pedidos mes

142,88 €

Ticket medio

3

Cientes nuevos

VENTAS ÚLTIMOS 7 DÍAS

171

1134

625

259

417

842

29

30

31

01

02

03

04

PEDIDOS POR ESTADO (MES)

Entregado

6

Enviado

5

Preparación en curso

3

Pago aceptado

1

Cancelado

1

STOCK BAJO (5 O MENOS) · 5 SIN STOCK

Producto #22

0 uds

Producto #21

0 uds

Producto #25

0 uds

Producto #24

0 uds

Producto #27

0 uds

Producto #26

1 uds

Producto #23

2 uds

Producto #15

3 uds

Producto #3

4 uds

Producto #2

5 uds

ÚLTIMOS PEDIDOS

#1079 116,20 €

Enviado

#1075 179,23 €

Preparación en curso

#1078 224,07 €

Enviado

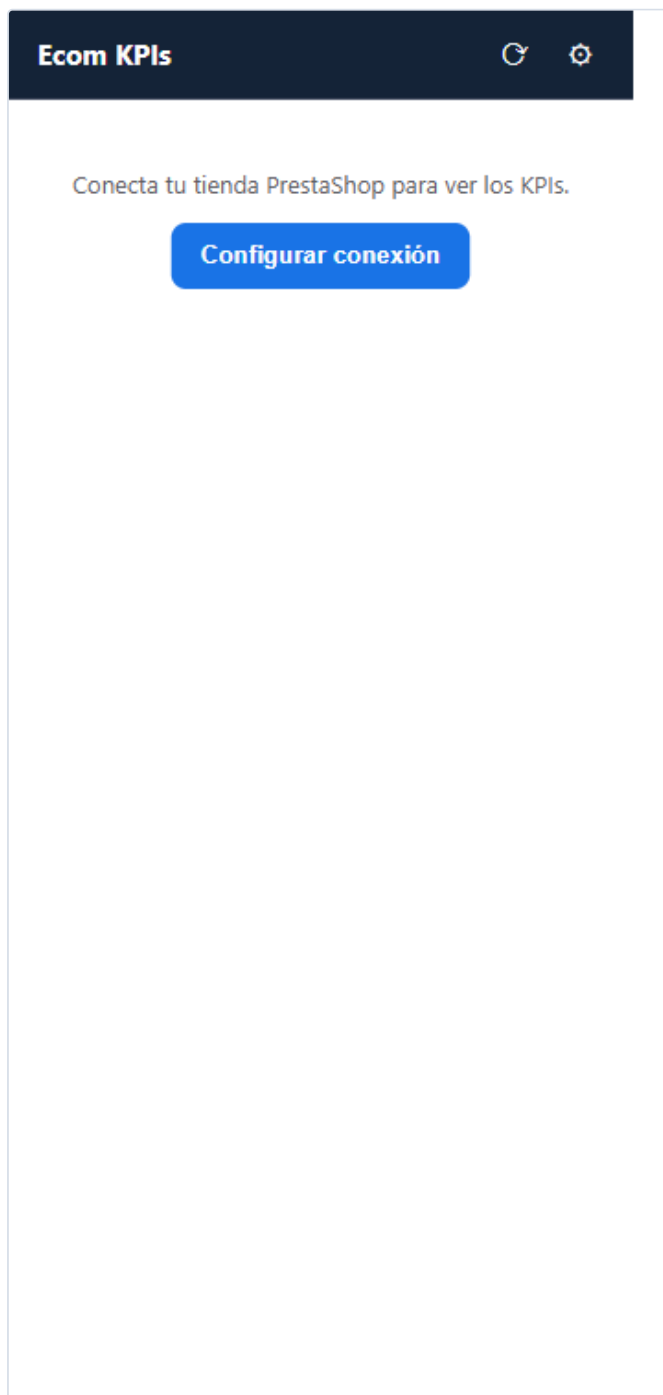
#1077 280,25 €

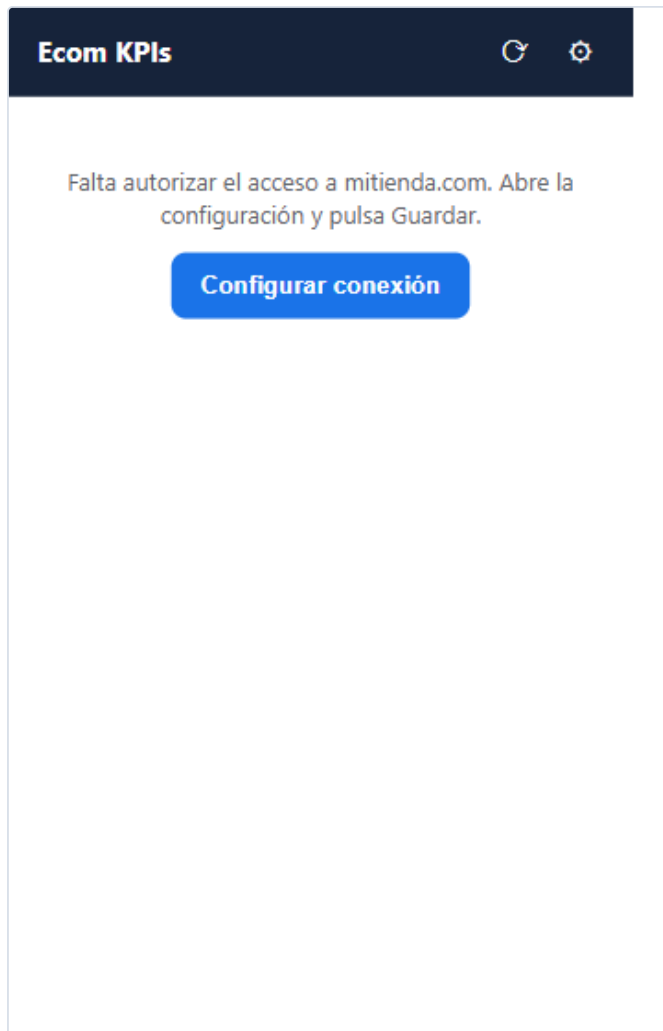
Entregado

#1071 280,33 €	entregado
#1076 42,20 €	Enviado
#1073 275,14 €	Pago aceptado
#1074 141,50 €	Entregado
#1072 54,10 €	Entregado

localhost:9457

Antes de configurar nada, y cuando falta el permiso del dominio:





Cuando la clave no vale:



6. Qué ve el cliente en la tienda

Nada. La extensión no inyecta ningún script en ninguna página, no modifica la tienda y solo hace peticiones GET de lectura. Es una herramienta del comerciante, invisible para el comprador.

7. Preguntas frecuentes y problemas

Las generales están en la sección 7 del manual principal. Estas son las de Firefox:

«He pulsado Guardar y no sale ningún aviso de permiso, y luego el popup dice que falta autorizar». Tienes la pantalla de opciones abierta dentro de `about:addons`. Ciérrala y ábrela desde el icono de la extensión (⚙) o desde el botón «Configurar conexión» del popup: así se abre en su propia pestaña, que es donde Firefox sí muestra el aviso.

«**Firefox me dice que la extensión requiere una versión más reciente**». Estás en Firefox 115 o anterior. Actualiza a la 116 como mínimo; antes de esa versión no existe el mecanismo de permisos opcionales que usa la extensión.

«**¿Por qué al instalarla ya no me pide acceso a todos los sitios?**» Porque ya no lo pide: ese era el fallo principal que se ha corregido. Ahora arranca sin acceso a nada y solo pide el dominio de tu tienda, cuando lo guardas.

«**He cambiado de dominio y ha dejado de funcionar**». El permiso es por dominio. Pon la URL nueva en la configuración y pulsa Guardar. Firefox pedirá el permiso del dominio nuevo. El anterior puedes retirarlo desde `about:addons` → la extensión → **Permisos**.

«**¿Puedo quitarle el permiso sin desinstalarla?**» Sí, en `about:addons` → la extensión → **Permisos**. La extensión se queda con la configuración guardada pero deja de poder leer la tienda, y el popup te avisará de que falta autorizar.

8. Seguridad

8.1 Punto de partida

Esta versión venía marcando **1.0.1 y no llevaba ninguno de los arreglos** de la revisión de seguridad que se hizo sobre la versión Chrome. Se comprobó fichero a fichero antes de tocar nada: `lib/ws.js`, `options/options.js`, `options/options.html`, `popup/popup.html` y `popup/popup.js` estaban exactamente en el estado anterior a los arreglos, y el manifiesto declaraba `"host_permissions": ["http:///*/*", "https:///*/*"]`.

Es decir: **la versión publicada en AMO tenía los mismos nueve problemas** que se corrigieron en la de Chrome, incluido el grave.

8.2 Lo que se ha portado en esta revisión

Los nueve arreglos, todos. El detalle del **porqué** de cada uno está en la sección 8.2 del manual principal; aquí va el resumen y lo que ha habido que cambiar para Firefox:

#	Arreglo	Fichero	¿Igual que en Chrome?
1	<code>host_permissions</code> a todo internet → <code>optional_host_permissions</code> + permiso por dominio	<code>manifest.json</code> , <code>lib/ws.js</code> , <code>options/options.js</code>	No , ver 8.3
2	<code>credentials: 'omit'</code> y <code>referrerPolicy: 'no-referrer'</code>	<code>lib/ws.js</code>	Sí
3	Clave por cabecera <code>Authorization</code> ; <code>ws_key</code> solo como respaldo tras un 401/403	<code>lib/ws.js</code>	Sí
4	Rechazo de <code>http://</code> en dominios reales y validación real de la URL	<code>lib/ws.js</code> , <code>options/options.js</code>	Sí
5	Clave enmascarada con botón Ver	<code>options/options.html</code> , <code>options/options.js</code>	Sí
6	Tiempo límite de 20 s con <code>AbortController</code>	<code>lib/ws.js</code>	Sí
7	Cero <code>innerHTML</code> : <code>replaceChildren()</code>	<code>popup/popup.js</code>	Sí
8	Política de seguridad de contenidos explícita	<code>manifest.json</code>	Sí
9	Fallo de datos: «Esta semana» y el gráfico de 7 días perdían los días del mes anterior	<code>popup/popup.js</code>	Sí

Se ha verificado que en la extensión no queda **ni una sola escritura de HTML** (`innerHTML`, `outerHTML`, `insertAdjacentHTML`, `document.write`), ni `eval()`, ni `new Function()`, ni receptores de mensajes abiertos, ni scripts de contenido.

8.3 Lo que NO se ha podido portar tal cual, y su equivalente en Firefox

Tres cosas del arreglo de Chrome no valen igual en Firefox. Se dicen porque son la parte que de verdad tenía riesgo de quedar rota:

a) `permissions.request()` no sobrevive a un `await`. En Chrome, la pantalla de opciones consultaba primero `permissions.contains()` y, solo si faltaba, llamaba a `permissions.request()`. En Firefox eso **no funciona**: la marca de «gesto del usuario» se pierde en cuanto hay un `await` por medio y `permissions.request()` responde «*may only be called from a user input handler*», con lo que el botón Guardar habría dejado de conceder el permiso —en silencio, porque el mensaje se ve en la consola y no en pantalla—. Se ha reescrito `options/options.js`: la validación de la URL y de la clave es ahora una función **síncrona** (`validar()`), y `permissions.request()` es la **primera operación asíncrona** del manejador del clic. No hace falta consultar antes: si el permiso ya está concedido, `request()` devuelve `true` sin mostrar ningún aviso. Hay una comprobación automática que verifica que no se cuela ningún `await` por delante.

b) `browser.*` devuelve promesas y `chrome.*` usa retrollamada. El código de Chrome llamaba a `chrome.permissions.contains(perm, callback)`. En Firefox, pasarle una retrollamada a un método del espacio `browser.*` puede hacer que el validador de esquemas rechace la llamada. `lib/ws.js` detecta ahora cuál de los dos existe y llama a cada uno como corresponde —promesa en Firefox, retrollamada en Chromium—, normalizando todo a promesa. Se ha probado con las dos ramas simuladas: `getConfig`, `hasPermission`, `requestPermission`, la validación de URL y la detección de `localhost` dan el mismo resultado en ambas.

c) `optional_host_permissions` **no existe antes de Firefox 116**. Por eso

`browser_specific_settings.gecko.strict_min_version` sube de `115.0` a `116.0`. Dejarlo en 115 habría sido peor que no arreglar nada: el manifiesto se acepta, la clave se ignora y la extensión se queda sin permisos para nada.

Además, `popup/popup.js` llama ahora a `WS.api.runtime.openOptionsPage()` en lugar de `chrome.runtime.openOptionsPage()`, para pasar por la misma capa.

8.4 Lo que queda pendiente, y por qué

Lo mismo que en la versión Chrome, con un matiz de Firefox:

- **La clave se guarda sin cifrar** en el almacenamiento local del navegador. Es una limitación de todas las extensiones: no hay almacén cifrado al que puedan acceder. La defensa correcta es la que ya se aplica: **usa una clave de solo lectura** y revócala desde PrestaShop si pierdes el control del equipo.
- **El respaldo por `ws_key` sigue existiendo** para los alojamientos que no reenvían la cabecera `Authorization` a PHP. Con `https://` obligatorio la clave va cifrada en tránsito; el riesgo que queda son los registros de tu propio servidor.
- **Nombres de producto en el bloque de stock y filtro por tienda en multitienda:** no se han tocado, cambian el comportamiento y no son cuestiones de seguridad.
- `optional_host_permissions` **incluye `http://**`**. Se mantiene solo para poder probar contra `localhost`; el código rechaza `http://` en cualquier dominio real, así que ese patrón nunca llega a solicitarse para una tienda de verdad.

8.5 Cómo se ha comprobado

- `diff -r` **completo contra la carpeta de la versión Chrome** antes y después de portar, para confirmar que el punto de partida era el mismo código y que al final solo quedan las diferencias propias de Firefox (manifiesto, README, la capa `browser / chrome`, el orden del permiso y dos textos de la pantalla de opciones).
- `node --check` sobre `lib/ws.js`, `popup/popup.js` y `options/options.js`, y validación del `manifest.json` como JSON.
- **Prueba automática de `lib/ws.js` con las dos ramas de API simuladas** (Firefox con promesas y Chromium con retrollamada): comprueba `normalizeUrl` (quita barras finales, rechaza `ftp://` y las direcciones mal escritas), `isLocal` (acepta `localhost`, rechaza un dominio real), `getConfig`, y que `hasPermission` / `requestPermission` distinguen el dominio concedido del que no lo está. Las dos ramas dan el mismo resultado.
- **Comprobación automática del orden del gesto del usuario** en `options/options.js`: que no haya ningún `await` antes de `permissions.request()` en los manejadores de **Guardar** y **Probar conexión**, y que no se llame a `permissions.contains()` por delante. Es el fallo que habría dejado el botón sin efecto en Firefox, y no lo caza ningún linter.
- **Búsqueda de patrones peligrosos** en toda la carpeta: cero `innerHTML`, cero `eval`, cero `chrome.*` directo fuera de la capa de compatibilidad.

No se ha podido cargar la extensión en un Firefox real con Playwright: Playwright no expone ninguna forma de instalar complementos en su Firefox. Las capturas de este manual son las de la versión Chrome, con la advertencia de la sección 4. Lo que queda sin probar en pantalla, y conviene mirar al publicar en AMO, es **el aviso de permiso de Firefox**: que aparezca al pulsar Guardar en la pestaña de opciones y que al aceptarlo el popup cargue los KPIs.

9. Ficha técnica

Versión: 1.0.2 (subida desde 1.0.1, que era el código sin los arreglos). **Manifiesto:** MV3. **Autor:** Ecom Experts.

Ficheros:

```
ecom_kpis_prestashop-firefox/
manifest.json      MV3, browser_specific_settings.gecko, CSP, permisos opcionales
lib/ws.js          cliente de la Webservice + capa browser/chrome + permisos
popup/popup.html   estructura del cuadro de mandos
popup/popup.css     estilos (idéntico a la versión Chrome)
popup/popup.js      cálculo de KPIs y pintado (solo textContent)
options/options.html pantalla de configuración
options/options.js  validación, permiso por dominio y guardado
icons/             16, 48 y 128 px
```

Permisos:

Clave	Valor	Para qué
<code>permissions</code>	<code>["storage"]</code>	guardar URL, clave, umbral y moneda
<code>optional_host_permissions</code>	<code>["https://*/*", "http://*/*"]</code>	pedir, uno a uno , el dominio de la tienda
<code>content_security_policy.extension_pages</code>	<code>script-src 'self'; object-src 'self'</code>	impedir código que no venga de la propia extensión
<code>browser_specific_settings.gecko.strict_min_version</code>	<code>116.0</code>	mínimo con <code>optional_host_permissions</code>
<code>browser_specific_settings.gecko.data_collection_permissions</code>	<code>{ "required": ["none"] }</code>	declaración a AMO: no se recogen datos

Recursos de la Webservice que consulta (solo GET): `orders`, `order_states`, `stock_availabilitys`, `customers`.

Red: una sola llamada, a la tienda configurada, con `credentials: 'omit'`, `referrerPolicy: 'no-referrer'`, `cache: 'no-store'` y corte a los 20 segundos. Sin telemetría, sin scripts de contenido, sin receptores de mensajes, sin proceso de fondo (no hay `background` / `service_worker`: todo ocurre mientras el popup o las opciones están abiertos).

Almacenamiento: `storage.local`, cuatro claves — `url`, `key`, `stock`, `currency`.

Registros: la extensión no escribe ningún fichero de registro. Los errores se ven en la consola de la ventana emergente (clic derecho sobre el popup → Inspeccionar).